K.R. Mangalam University

School of Engineering & Technology

DESIGN AND ANALYSIS OF ALGORITHMS LAB MANUAL

Program: BCA (AI & DS)

Course Code - ENCA301 (2025-2026)

Submitted by:

Name: Bhavya Rattan

Roll Number: 2401201004

Course: BCA (AI & DS) - Section B

Submitted To:

Dr. Aarti Sangwan

Assistant Professor

5. Complexity Table

Algorithm	Best	Average	Worst	Space
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)
Fibonacci Recursive	$O(2^n)$	$O(2^n)$	$O(2^n)$	O(n)
Fibonacci Iterative	O(n)	O(n)	O(n)	O(1)
Fibonacci DP	O(n)	O(n)	O(n)	O(n)

Description

The complexity table shows the theoretical efficiency of all algorithms analysed in this experiment. Bubble Sort and Insertion Sort have $O(n^2)$average and worst-case time complexity, which explains their slower performance for large input sizes. Merge Sort provides consistent $O(n \log n)$performance in all cases but requires additional $O(n)$memory.
Quick Sort has an average complexity of $O(n \log n)$and performed efficiently in the experimental results, although its worst-case complexity can become $O(n^2)$.
For Fibonacci computation, Naive Recursive Fibonacci has exponential $O(2^n)$time complexity due to repeated calculations. Iterative Fibonacci is more efficient with $O(n)$time and $O(1)$space. Dynamic Programming also achieves $O(n)$time but uses $O(n)$memory to store previously calculated values.